

10) Smart Energy Management Using Reinforcement Learning

Aim: Use Reinforcement Learning (Q-Learning) to keep electricity consumption below a specified threshold

CODE:

```
import numpy as np
import random

# Parameters
MAX_CONSUMPTION = 100
THRESHOLD = 70

ACTIONS = [-5, 0, 5] # Reduce, Maintain, Increase

ALPHA = 0.1
GAMMA = 0.9
EPSILON = 0.2

EPISODES = 500

# Q Table
Q = np.zeros((MAX_CONSUMPTION + 1, len(ACTIONS)))

def choose_action(state):
    if random.uniform(0,1) < EPSILON:
        return random.randint(0,2)
    return np.argmax(Q[state])

for episode in range(EPISODES):

    state = random.randint(40,90)

    for step in range(50):

        action = choose_action(state)

        new_state = state + ACTIONS[action]

        new_state = max(0,min(MAX_CONSUMPTION,new_state))

        # Reward Function
        if new_state <= THRESHOLD:
            reward = 10 - abs(THRESHOLD-new_state)*0.1
        else:
            reward = -20

        # Q-learning Update
        Q[state,action] = Q[state,action] + ALPHA * (
            reward + GAMMA*np.max(Q[new_state]) - Q[state,action]
        )

        state = new_state
```

```
print("\nTraining Completed\n")
```

```
# Testing
```

```
state = 85
```

```
print("Initial Consumption:",state)
```

```
for i in range(15):
```

```
    action = np.argmax(Q[state])
```

```
    state = state + ACTIONS[action]
```

```
    state = max(0,min(MAX_CONSUMPTION,state))
```

```
    print("Step",i+1,  
          " Action:",ACTIONS[action],  
          " Consumption:",state)
```

```
OUTPUT:
```

```
===== RESTART: C:/Users/harik/Downloads/E10.py =====  
  
Training Completed  
  
Initial Consumption: 85  
Step 1 Action: -5 Consumption: 80  
Step 2 Action: -5 Consumption: 75  
Step 3 Action: -5 Consumption: 70  
Step 4 Action: 0 Consumption: 70  
Step 5 Action: 0 Consumption: 70  
Step 6 Action: 0 Consumption: 70  
Step 7 Action: 0 Consumption: 70  
Step 8 Action: 0 Consumption: 70  
Step 9 Action: 0 Consumption: 70  
Step 10 Action: 0 Consumption: 70  
Step 11 Action: 0 Consumption: 70  
Step 12 Action: 0 Consumption: 70  
Step 13 Action: 0 Consumption: 70  
Step 14 Action: 0 Consumption: 70  
Step 15 Action: 0 Consumption: 70  
>>
```

6) Reinforcement Learning for Traffic Signal Control

Aim: Implement an RL agent to control traffic signals while giving immediate priority to emergency vehicles.

Reinforcement Learning (Q-Learning)

Traffic Signal Control with Emergency Vehicle Priority

```
import numpy as np
```

```
import random
```

```
# Parameters
```

```
states = 2          # 0 = Normal Traffic, 1 = Emergency Vehicle
```

```
actions = 2         # 0 = North Green, 1 = East Green
```

```
alpha = 0.1         # Learning Rate
```

```
gamma = 0.9         # Discount Factor
```

```
epsilon = 0.2       # Exploration Rate
```

```
episodes = 500
```

```
# Q-Table
```

```
Q = np.zeros((states, actions))
```

```
# Training
```

```
for episode in range(episodes):
```

```
    state = random.randint(0, 1)
```

```
    for step in range(10):
```

```
        # Safety Constraint
```

```
        if state == 1:
```

```
            action = 0          # Emergency lane gets Green
```

```
            reward = 100
```

```
            next_state = 0
```

```
        else:
```

```
            # Epsilon-Greedy Action Selection
```

```
            if random.uniform(0, 1) < epsilon:
```

```
                action = random.randint(0, actions - 1)
```

```
            else:
```

```
                action = np.argmax(Q[state])
```

```
        # Reward
```

```
        reward = 10
```

```
        # Random next state
```

```
        next_state = random.randint(0, 1)
```

```
# Q-Learning Update
```

```
Q[state][action] = Q[state][action] + alpha * (
    reward + gamma * np.max(Q[next_state]) - Q[state][action]
```

```

    )

    state = next_state

print("Training Completed\n")

print("Q-Table:")
print(Q)

# Testing
print("\nTesting Agent\n")

for i in range(10):

    state = random.randint(0, 1)

    if state == 1:
        print(f"Step {i+1}: Emergency Vehicle Detected")
        print("Signal: Green for Emergency Lane")
    else:
        action = np.argmax(Q[state])

        if action == 0:
            print(f"Step {i+1}: Normal Traffic")
            print("Signal: North Green")
        else:
            print(f"Step {i+1}: Normal Traffic")
            print("Signal: East Green")

print("-" * 30)
===== RESTART: C:/Users/harik/Downloads/E10.py =====
Training Completed

Q-Table:
[[373.66002416 383.24649646]
 [449.86294843  0.          ]]

Testing Agent

Step 1: Emergency Vehicle Detected
Signal: Green for Emergency Lane
-----
Step 2: Emergency Vehicle Detected
Signal: Green for Emergency Lane
-----
Step 3: Normal Traffic
Signal: East Green
-----
Step 4: Emergency Vehicle Detected
Signal: Green for Emergency Lane
-----
Step 5: Emergency Vehicle Detected
Signal: Green for Emergency Lane
-----
Step 6: Normal Traffic
Signal: East Green
-----
Step 7: Emergency Vehicle Detected
Signal: Green for Emergency Lane
-----
Step 8: Emergency Vehicle Detected
Signal: Green for Emergency Lane
-----
Step 9: Normal Traffic
Signal: East Green
-----
Step 10: Emergency Vehicle Detected
Signal: Green for Emergency Lane
-----
>

```

9) Reinforcement Learning Framework for Delivery Drone

Aim

Design a Reinforcement Learning (Q-Learning) framework for a delivery drone that maximizes successful deliveries while avoiding no-fly zones and maintaining battery limits.

```
import numpy as np
import random

# Parameters
states = 5          # Drone locations
actions = 4         # 0=Up, 1=Down, 2=Left, 3=Right

alpha = 0.1
gamma = 0.9
epsilon = 0.2
episodes = 500

Q = np.zeros((states, actions))

for episode in range(episodes):

    state = random.randint(0, states - 1)
    battery = 100

    for step in range(10):

        # Choose action (Epsilon-Greedy)
        if random.uniform(0, 1) < epsilon:
            action = random.randint(0, actions - 1)
        else:
            action = np.argmax(Q[state])

        next_state = random.randint(0, states - 1)
        battery -= 10

        # Rewards
        if next_state == 4:
            reward = 100      # Successful delivery
        elif next_state == 2:
            reward = -100     # No-fly zone
        elif battery <= 20:
            reward = -50      # Low battery
        else:
            reward = 10

        # Bellman Update
        Q[state][action] += alpha * (
            reward + gamma * np.max(Q[next_state]) - Q[state][action]
        )

    state = next_state
```

```

    if battery <= 0:
        break

print("Training Completed\n")
print("Q-Table:\n", Q)

===== RESTART: C:/Users/harik/Downloads/E10.py =====
Training Completed

Q-Table:
[[-31.37221528 -33.0365033 -23.10786849 -27.12674904]
 [-34.03841844 -25.07418841 -35.05924649 -9.4223862 ]
 [-32.24428813 -40.32171107 -7.15975705 -37.46974693]
 [-11.17642663 -27.22588176 -25.78400609 -36.16036634]
 [-31.52974936 -61.59434404 -31.48324858 -21.01667188]]
.>>|

```

3) Markov Decision Process (MDP) for an Autonomous Robot

Aim

Design an MDP for a robot that reaches a destination while minimizing energy consumption.

Markov Decision Process (MDP) for Autonomous Robot

```
import random
```

```
# States
```

```
states = ["Start", "A", "B", "C", "Goal"]
```

```
# Actions
```

```
actions = ["Up", "Down", "Left", "Right"]
```

```
# Initial State
```

```
state = "Start"
```

```
# Battery
```

```
battery = 100
```

```
# Rewards
```

```
goal_reward = 100
```

```
move_cost = -10
```

```
obstacle_penalty = -50
```

```
print("Robot Navigation\n")
```

```
while state != "Goal" and battery > 0:
```

```
    action = random.choice(actions)
```

```
    # Transition Probability
```

```
    if random.random() < 0.9:
```

```
        # Successful movement
```

```
        if state == "Start":
```

```

        state = "A"
    elif state == "A":
        state = "B"
    elif state == "B":
        state = "C"
    elif state == "C":
        state = "Goal"

    reward = move_cost

else:
    # Obstacle encountered
    reward = obstacle_penalty

battery -= 10

print("Action:", action)
print("State:", state)
print("Reward:", reward)
print("Battery:", battery)
print("-----")

# Final Result
if state == "Goal":
    print("Destination Reached!")
    print("Final Reward:", goal_reward)
else:
    print("Battery Exhausted!")

```

```

>>> [ 01.02574990  01.02574990  01.40024000  21.01007100]]
===== RESTART: C:/Users/harik/Downloads/E10.py =====
Robot Navigation

Action: Left
State: A
Reward: -10
Battery: 90
-----
Action: Right
State: B
Reward: -10
Battery: 80
-----
Action: Right
State: C
Reward: -10
Battery: 70
-----
Action: Down
State: Goal
Reward: -10
Battery: 60
-----
Destination Reached!
Final Reward: 100

```

1) Reinforcement Learning for Grid World Navigation Using OpenAI Gym

```
import numpy as np
import random

# -----
# Grid World Environment
# -----

GRID_SIZE = 5

START = (0, 0)
GOAL = (4, 4)

OBSTACLES = [
    (0, 3),
    (1, 1),
    (2, 3),
    (3, 0),
    (4, 2)
]

ACTIONS = {
    0: (-1, 0), # Up
    1: (1, 0), # Down
    2: (0, -1), # Left
    3: (0, 1) # Right
}

# -----
# Q-Learning Parameters
# -----

ALPHA = 0.1
GAMMA = 0.9
EPSILON = 0.2

EPISODES = 500
MAX_STEPS = 100

Q = np.zeros((GRID_SIZE * GRID_SIZE, 4))

def state_to_index(state):
    return state[0] * GRID_SIZE + state[1]

def step(state, action):
    row, col = state
```



```

dr, dc = ACTIONS[action]

new_row = row + dr
new_col = col + dc

# Boundary check
if new_row < 0 or new_row >= GRID_SIZE or new_col < 0 or new_col >= GRID_SIZE:
    return state, -10, False

new_state = (new_row, new_col)

# Obstacle check
if new_state in OBSTACLES:
    return state, -50, False

# Goal check
if new_state == GOAL:
    return new_state, 100, True

return new_state, -1, False

# -----
# Training
# -----

for episode in range(EPISODES):

    state = START

    done = False

    for _ in range(MAX_STEPS):

        s = state_to_index(state)

        if random.uniform(0, 1) < EPSILON:
            action = random.randint(0, 3)
        else:
            action = np.argmax(Q[s])

        next_state, reward, done = step(state, action)

        ns = state_to_index(next_state)

        Q[s, action] = Q[s, action] + ALPHA * (
            reward + GAMMA * np.max(Q[ns]) - Q[s, action]
        )

        state = next_state

```

```

        if done:
            break

print("=====")
print("Training Completed Successfully")
print("=====")

# -----
# Test Learned Policy
# -----

print("\nOptimal Path:\n")

state = START

path = [state]

visited = set()

for _ in range(MAX_STEPS):

    if state == GOAL:
        break

    visited.add(state)

    s = state_to_index(state)

    action = np.argmax(Q[s])

    next_state, reward, done = step(state, action)

    if next_state == state:
        break

    if next_state in visited:
        break

    path.append(next_state)

    state = next_state

    if done:
        break

print(path)

# -----
# Display Grid

```

```
# -----

print("\nGrid World:\n")

grid = [["." for _ in range(GRID_SIZE)] for _ in range(GRID_SIZE)]

for r, c in OBSTACLES:
    grid[r][c] = "X"

grid[START[0]][START[1]] = "S"
grid[GOAL[0]][GOAL[1]] = "G"

for r, c in path:
    if (r, c) != START and (r, c) != GOAL:
        grid[r][c] = "*"

for row in grid:
    print(" ".join(row))
```

```
===== RESTART: C:/Users/narik/Downloads/E10.py =====
Training Completed Successfully
=====

Optimal Path:

[(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (3, 2), (3, 3), (3, 4), (4, 4)]

Grid World:

S * * X .
. X * . .
. . * X .
X . * * *
. . X . G

>>> |
```